11 Caching data in memory: Amazon ElastiCache and MemoryDB

This chapter covers

- The benefits of a having a caching layer between your application and data store
- Defining key terminology, such as cache cluster, node, shard, replication group, and node group
- Using /operating an in-memory key-value store
- Performance tweaking and monitoring ElastiCache clusters

Imagine a relational database being used for a popular mobile game where players' scores and ranks are updated and read frequently. The read and write pressure to the database will be extremely high, especially when ranking scores across millions of players. Mitigating that pressure by scaling the database may help with load but not necessarily the latency or cost. Also, caching relational databases tends to be more expensive than caching data stores.

A proven solution used by many gaming companies is leveraging an in-memory data store such as Redis for both caching and ranking player and game metadata. Instead of reading and sorting the leaderboard directly from the relational database, they store an in-memory game leaderboard in Redis, commonly using a sorted set, which will sort the data automatically when it's inserted, based on the score parameter. The score value may consist of the actual player ranking or player score in the game.

Because the data resides in memory and does not require heavy computation to sort, retrieving the information is incredibly fast, leaving little reason to query directly from the relational database. In addition, any other game and player metadata, such as player profile, game-level information, and so on, that requires heavy reads can also be cached using this in-memory layer, thus alleviating heavy read traffic to and from the database.

In this solution, both the relational database and in-memory layer will store updates to the leaderboard: one will serve as the primary database and the other as the